Semana 1 Sesión 1

Sistemas embebidos

Los sistemas embebidos están dentro de otro aparato. Ej: lavadora, control remoto, alarma. No es como la PC, ya está destinado a una función.

Un microprocesador es básicamente la CPU, necesita conectarse con RAM y demás componentes. El microcontrolador tiene dentro la CPU, memorias y los periféricos, por eso es más pequeño y se usa mucho en controles.

Computador normal: varias cosas al mismo tiempo. El embebido hace lo que necesita el equipo, normalmente con menos recursos.

Dentro del microcontrolador

CPU ejecuta las instrucciones, memoria guarda el programa o datos. Todo se conecta por buses.

ALU: operaciones de suma, resta, AND, OR, XOR.

Registros: como casilleros dentro de CPU, muy rápidos. Unidad de control decide qué instrucción toca. PC o contador de programa guarda dirección de la siguiente instrucción.

El reloj es el que marca el ritmo. La CPU hace buscar, decodificar y ejecutar, una y otra vez.

Buscar toma instrucción, decodificar interpreta y ejecutar hace la operación.

16 MHz = 16 millones de ciclos cada segundo. 1 ciclo = 62,5 ns, pero una instrucción puede demorar varios ciclos.

El ATmega tiene un núcleo y el ESP32 clásico dos. Más núcleos permiten repartir trabajo, no significa que siempre va a ser el doble de rápido.

Los buses

Direcciones = dónde está la información.

Datos = la información que se manda.

Control = dice si toca leer o escribir.

Si quiero escribir en 0x25 el dato 0x20, la CPU coloca dirección, el dato y la señal de escritura.

Para leer es al revés, se pide la dirección y el dispositivo devuelve el valor.

Hay buses que van en un solo sentido y otros en dos. El bus de datos normalmente necesita poder leer y escribir.

Von Neumann usa un camino compartido para programa y datos; se turnan. Harvard separa instrucciones de datos, pueden acceder por diferentes buses. ATmega328P tiene arquitectura Harvard.

Se hablaba del cuello de botella cuando todo debe pasar por el mismo camino

Memorias

Flash guarda el programa aunque se apague. SRAM es para variables mientras está funcionando y pierde todo sin energía. EEPROM para valores que deben quedar guardados, como una configuración.

Flash y EEPROM tienen límite de escrituras, leer no las desgasta igual. Por eso no escribir EEPROM dentro de loop cada vuelta.

1 byte = 8 bits. El ATmega328P tiene 32 KB de Flash, 2 KB SRAM y 1 KB EEPROM. El ESP32 del ejemplo tiene aprox. 520 KB de SRAM interna y módulo con Flash de varios MB.

La RAM guarda los datos de ahora. Una lectura de sensor que cambia a cada momento

Los flip-flops pueden almacenar bits. Hay memorias hechas con diferentes circuitos; no todas usan lo mismo.

Periféricos

Son circuitos del microcontrolador que hacen otras tareas sin que la CPU esté pendiente de cada detalle.

GPIO para entradas y salidas. ADC pasa de voltaje a número. PWM manda pulsos. Timer cuenta tiempo. Interrupción avisa cuando pasó un evento. UART, SPI e I2C sirven para comunicar otros componentes.

ATmega328P / Arduino Uno: 8 bits, 16 MHz, pines de 5 V. ESP32: 32 bits, Wi-Fi y Bluetooth, pines de 3,3 V. No mandar 5 V directo a sus GPIO.

Chip ESP32 no es toda la placa. WROOM es módulo con chip y otros componentes, DevKit es la placa con USB, regulador y botones.

Para escoger uno hay que revisar los pines que necesito, memoria, consumo, Wi-Fi y cómo se va a alimentar. No siempre conviene usar el más potente.

Quedó revisar lo del ejercicio con buses, distinguir bien qué va en cada uno.

Semana 1 Sesión 2

Bits y hexadecimal

El byte son 8 bits. Se cuentan del 0 al 7 empezando por la derecha. Los pesos son 1, 2, 4, 8, 16, 32, 64, 128.

00100000 = 32, en hexadecimal 0x20.

10000001 = 129 = 0x81.

0xFF vale 255 y 0x08 señala el bit 3.

Hexadecimal: 0 al 9, después A, B, C, D, E, F. Cada dígito hex ocupa 4 bits. Sirve mucho para escribir registros sin poner tantos ceros.

Uno suele manejar 5 V como nivel alto, ESP32 3,3 V. Son niveles lógicos, no es que el chip solo conozca 0 V y el voltaje exacto.

Operaciones con bits

AND: tienen que ser 1 los dos. OR: basta que uno sea 1. XOR: uno diferente del otro. NOT invierte.

0xCA & 0x0F = 0x0A, se queda con los cuatro bits de abajo.

Para seleccionar una posición usamos máscara. 1 << 5 da 00100000, osea bit 5.

REG |= (1 << n) para prender un bit.

REG &= ~(1 << n) para apagarlo.

REG ^= (1 << n) lo cambia.

REG & (1 << n) para ver si está activo.

Ojo con REG = (1 << 2), ahí reemplazo todo el valor y se me borran los otros bits. Con |= conservo los que ya estaban.

Si el registro era 10101010 y debo cambiar solo una posición, no es necesario escribir los 8 bits otra vez.

Registros

Una dirección especial de memoria puede servir para controlar un periférico. Escribir ahí realmente puede hacer que un pin cambie.

En AVR el programa C termina convertido en instrucciones máquina. LDI carga, ADD suma, AND, OR, EOR; SBI pone un bit y CBI lo borra. RJMP hace salto.

RISC instrucciones simples, CISC más complejas. El AVR es de 8 bits, entonces cuando hay números mayores debe hacer más operaciones.

PORTB |= (1 << 5) en el ejemplo puede convertirse en una instrucción SBI. No es lo mismo escribir el registro entero.

DDR, PORT, PIN

DDR decide la dirección del pin. 1 significa salida, 0 entrada.

Cuando es salida, PORT con 1 lo pone HIGH, con 0 LOW. Si es entrada, PORT con 1 activa la resistencia pull-up interna del AVR. PIN sirve para leer el valor que tiene el pin.

DDRB5 = 1, el PB5 trabaja como salida. PORTB5 = 1 entonces sale alto.

Si DDR=0 y PORT=0 queda como entrada sin pull-up. Si nadie conecta nada, el valor puede quedar flotando.

DDR=0 y PORT=1 es entrada con pull-up. El botón a GND, al soltar se lee 1; cuando se presiona se lee 0. La lógica es invertida.

Entrada flotante: no es un cero fijo ni un uno fijo. Puede cambiar por ruido.

Para un LED hace falta resistencia, no conectarlo así nomás.

Con Arduino: pinMode(13, OUTPUT) en Uno configura DDRB5. digitalWrite(13, HIGH) modifica PORTB5. INPUT_PULLUP deja entrada y habilita resistencia.

La diferencia es que Arduino lo hace con funciones y el registro se puede manejar directamente.

En un ejercicio: 0x81 = 10000001. Para quitar bit 4, PORTB &= ~(1 << 4). Si el botón usa pull-up y está presionado, lo que se lee

También preguntar por qué un cambio con |= conserva otros bits y con = no.

Semana 2 Sesión 1

ESP32 y los registros

Lo de AVR era DDR, PORT y PIN. En el ESP32 cambia el nombre, pero se sigue controlando el hardware escribiendo bits.

ATmega trabaja con 8 bits, ESP32 clásico 32. Tiene dos núcleos, más memoria y comunicación inalámbrica. El voltaje de los GPIO es 3,3 V.

La placa DevKit tiene el módulo WROOM, chip USB-serie (puede ser CP2102 o CH340), regulador, botón EN y BOOT. El WROOM no es lo mismo que toda la placa.

GPIO significa entrada/salida de propósito general. Con la GPIO matrix varias señales de los periféricos pueden dirigirse a diferentes pines. IO_MUX configura las funciones.

GPIO_ENABLE parecido al DDR para configurar salida. GPIO_OUT maneja lo que se escribe y GPIO_IN para leer.

W1TS = write 1 to set, pone a 1 los bits que le indico.

W1TC = write 1 to clear, limpia esos bits. Los ceros que escriba no afectan las otras posiciones.

Si uso leer-modificar-escribir un registro, puede venir otra tarea o interrupción y cambiarlo antes de que guarde. Ahí se pierde el cambio. W1TS y W1TC ayudan porque se escribe directamente la máscara.

Ejemplo GPIO16, máscara 1UL << 16. Para prender con GPIO_OUT_W1TS y para apagar con W1TC. Antes debe estar configurado como salida.

En Arduino digitalWrite hace todo más fácil, pero debajo llama otras partes hasta llegar al registro.

Cuidado con los pines

GPIO16, 17, 18, 19, 21, 22, 23 suelen servir para prácticas, viendo que estén libres.

GPIO34 al 39 solamente entrada y no tienen pull-up interna. Los 6 al 11 normalmente van conectados con la memoria Flash del módulo, no ocuparlos.

GPIO1 y 3 usados por UART0, por ahí puede ir el monitor serie o la carga.

Los strapping son pines que el ESP32 lee al encender. GPIO0 puede decidir modo de programación. También 2, 5, 12 y 15, tener cuidado con lo que se conecta.

Si un sensor mantiene una condición equivocada cuando el chip arranca

GPIO35 para botón sí se puede usar como entrada, pero necesita resistencia externa si quiero pull-up. GPIO18 para LED suele ser más sencillo.

La fuente USB de la placa puede ser 5 V, pero los GPIO NO toleran 5 V. Resistencias en los LEDs; el cálculo era R = (3,3 - voltaje del LED) / corriente.

Del reset al loop

Primero el arranque desde ROM y bootloader, luego inicia la aplicación. ESP32 con Arduino usa funciones del core y FreeRTOS por debajo.

setup() corre una vez, sirve para configurar pines, serial, etc. loop() se repite. Es un superlazo, no se termina al acabar la primera vuelta.

En una vuelta se leen entradas, se decide y se actualizan salidas. Y otra vez lo mismo.

Variable global conserva el dato mientras corre el programa. Local normal dentro de loop se vuelve a crear. Con static el valor queda guardado entre vueltas.

Ejemplo contador: global 1,2,3; local 1,1,1; static 1,2,3.

La CPU sigue trabajando aunque el loop casi no haga nada. Si se quiere ahorrar batería, modos de bajo consumo o deep sleep, dependiendo del equipo.

FreeRTOS organiza tareas. Si uso el core de Arduino, no debo poner otro app_main() porque puede chocar con lo que ya trae.

Para cargar el programa

Arduino IDE, instalar esp32 by Espressif Systems. Elegir placa y puerto. El cable tiene que pasar datos, no solo carga.

Si no aparece puerto revisar driver; si no conecta, BOOT y strapping. Brownout cuando la alimentación cae. El puerto también puede estar ocupado.

En Wokwi se puede practicar el Blink con GPIO16, pero la placa física puede cambiar en sus conexiones.

La configuración de entrada y salida depende del GPIO y del módulo que estoy usando.

Semana 2 Sesión 2

Tipos de datos

Una variable tiene bits pero el tipo dice cómo interpretarlos. 11001100 en uint8_t es 204. Si fuera int8_t es -52.

En Arduino Uno el int normalmente ocupa 2 bytes, en ESP32 ocupa 4. Por eso cuando se necesita el tamaño exacto se usa int8_t, uint8_t, uint16_t, uint32_t.

uint8_t va de 0 a 255. Cuando llega a 255 y sumo 1, vuelve a 0. Los enteros sin signo trabajan dando la vuelta.

Con 200 + 100 el resultado matemático es 300, pero guardado en uint8_t queda 44.

No hay que confiar en el overflow de los tipos con signo. sizeof() sirve para saber cuantos bytes ocupa un tipo.

Cuando hago ~a siendo a de 8 bits, primero puede convertirse a int para la operación. Por eso a veces el resultado impreso tiene muchos bits. Se puede convertir a uint8_t si quiero conservar solo los 8.

Para el bit 31 del ESP32, mejor 1UL << 31, por el asunto del signo del int.

const para valores que no deben cambiar, constexpr se calcula como constante cuando aplica.

#define reemplaza texto y enum para nombres de estados.

static en una función hace que la variable conserve valor entre llamadas. volatile se usa cuando la variable puede cambiar desde una interrupción o hardware; no significa que ya sea atómica.

Un error: uint8_t i y condición i < 256, porque nunca va a llegar a 256. Otro error guardar millis() en int.

Tiempo sin delay

Con delay(500), el programa está esperando y no pregunta por el botón durante ese tiempo. Si alguien presiona rápido, no alcanza a verlo.

millis() da tiempo desde que empezó el programa, en milisegundos. Guardarlo en uint32_t o unsigned long según la placa.

ahora = millis(); si (ahora - anterior >= intervalo) hago la tarea y guardo el tiempo.

La resta es importante, porque cuando millis da la vuelta sigue funcionando si se usan tipos sin signo e intervalos adecuados.

Con dos LEDs: uno cambia cada 500 ms y otro cada 130 ms. Cada uno necesita su anterior y su estado, así pueden andar al mismo tiempo sin quedarse esperando.

No es que las dos instrucciones se ejecutan exactamente en paralelo, el loop pasa rápido revisando cada condición.

Pulsadores

Un botón sin resistencia puede dejar la entrada flotando. INPUT_PULLUP mantiene el pin en alto cuando nadie toca el botón.

Botón entre GPIO y GND: sin presionar HIGH, presionado LOW. Parece al revés pero así funciona con pull-up.

GPIO34-39 del ESP32 clásico no llevan pull-up interna. Si uso esos pines debo colocar la resistencia externamente.

El rebote es cuando al presionar, el contacto hace varios 0 y 1 en milisegundos, aunque la persona lo tocó solo una vez.

Antirrebote: aceptar el cambio cuando el estado se mantiene estable 20 o 30 ms. Evitar un delay grande porque otra vez se pierde lo que pasa.

Nivel: sigue presionado. Flanco: acaba de cambiar de HIGH a LOW. Para contar pulsaciones debo detectar el flanco y no sumar cada vuelta que siga en LOW.

Guardar lectura anterior, estado estable y el tiempo del último cambio. Si vuelve a cambiar rápido, esperar.

En el juego de luces se separa leer botón, actualizar posición/velocidad y mostrar LED. El botón puede cambiar velocidad mientras siguen las luces.

Cuando se lleva mucho tiempo presionado, no debería contar muchas veces

La idea de todo esto es que loop siga girando y que ninguna tarea lo deje esperando demasiado.

Semana 3

Temporizadores

GPIO18 no es el pin físico número 18 de la placa. Los GPIO son nombres de las señales, en el DevKit pueden estar en otra posición.

Recordar: GPIO34 al 39 solamente entrada. Y una variable static dentro de loop guarda el valor entre vueltas.

El timer es un periférico de hardware que cuenta pulsos de reloj. No es un for que esté sumando con la CPU, el circuito cuenta mientras el programa puede hacer otra cosa.

Si el loop usa millis() tiene que ir preguntando la hora. Con un temporizador de hardware se puede configurar una alarma y recibir el aviso.

Tiene registros de control, contador y alarma. La CPU lo inicia pero después los ticks se van sumando en el propio periférico.

El reloj y el divisor

En el ejemplo del ESP32 clásico, el reloj APB es 80 MHz. Son 80 millones de pulsos en un segundo, cada pulso equivale a 12,5 ns.

Prescaler es el divisor. Si N = 80, entonces 80 MHz / 80 = 1 MHz.

A 1 MHz, cada cuenta o tick dura 1 microsegundo.

Resolución = 1 / frecuencia de tick. Si divido más, cada tick dura más tiempo, osea menos resolución.

No confundir frecuencia del CPU con frecuencia del timer, son relojes y divisores que pueden estar configurados diferente.

El número de alarma A es hasta dónde quiero contar.

Tiempo = alarma / frecuencia de tick.

Alarma = tiempo × frecuencia de tick.

Si quiero 0,5 s con timer de 1 MHz, alarma 500000. Son 500000 microsegundos.

2000 ticks con 1 MHz son 2 ms.

10 ms necesitan 10000 ticks.

Con 10 kHz y alarma 2500: 0,25 s, da 4 eventos por segundo.

Otro ejercicio: frecuencia de 40 kHz y alarma 40000 = 1 segundo. Desde 80 MHz el prescaler sería 2000.

Con las unidades hay que fijarse si está en MHz, kHz, ms o µs. Ahí es donde uno se equivoca.

Alarma y desbordamiento

Autorrecarga: al llegar a la alarma vuelve a arrancar de la forma configurada y así se repite cada intervalo.

Desbordamiento es diferente: el contador llegó hasta el número más grande que puede representar. En 8 bits el máximo es 255, en 16 bits 65535.

La alarma la eligió el programa, el overflow viene por el tamaño del contador.

ESP32 clásico con TIMG0 y TIMG1, dos temporizadores por grupo, en total cuatro timers generales de 64 bits.

ATmega328P tiene Timer0 y Timer2 de 8 bits, Timer1 de 16 bits. Cambia la cantidad de bits y los prescalers.

En estos timers el contador puede ser grande, entonces el problema casi nunca es que llegue al límite en ejercicios pequeños.

Usar el timer desde Arduino

Una API son funciones que nos dejan utilizar el hardware sin escribir todos los registros. El core es el paquete que se instala para que Arduino pueda compilar y cargar código de esa placa.

La API del ESP32 cambió de la versión 2.x a 3.x. Por eso hay ejemplos viejos de temporizadores que no compilan con el core nuevo.

Con core 3.x hay que ver bien las funciones para crear timer, poner alarma y registrar interrupción. No copiar mezclando versiones.

Cuando llega a la alarma, el hardware puede generar interrupción y la CPU ejecuta una ISR.

Después regresa a lo que estaba haciendo.

ISR = rutina de interrupción. No debería durar mucho porque el programa deja de hacer otras tareas mientras atiende ese aviso.

En vez de imprimir ahí, mejor subir una bandera y que loop haga Serial.println o encienda el LED.

IRAM_ATTR tiene que ver con tener el código disponible en RAM de instrucciones. volatile para una variable que cambia fuera del flujo normal, pero tampoco protege por sí solo varias operaciones a la vez.

Si un temporizador tiene resolución de 1 µs no significa que el código de la ISR se ejecute exactamente al microsegundo, puede haber latencia.

El contador sigue avanzando aunque el loop esté ocupado. La parte que atiende la interrupción depende de las prioridades y del estado del sistema.

Quedó revisar la fórmula y distinguir alarma de overflow.

Semana 4

Interrupciones externas

Lo del botón de la semana pasada era sondeo. El loop revisaba digitalRead cada vuelta, o sea está preguntando si ya tocaron.

Si tiene un delay de 2 segundos, en ese rato puede presionar alguien y el programa ni se entera. El toque fue antes de que vuelva a leer.

Interrupción: el GPIO avisa al procesador cuando cambia el pin. No es necesario estar revisando desde loop.

La CPU deja un momento lo que estaba haciendo, se ejecuta la ISR y luego continúa. El timer también hacía interrupciones, pero ese está dentro del chip.

ISR = función que atiende el aviso.

No es igual que saludar(); porque no la estoy llamando desde mi código.

Nivel y flanco

Nivel: se mantiene en 1 o en 0

Flanco: cuando cambia

RISING 0 → 1

FALLING 1 → 0

CHANGE cuando cambia a cualquiera de los dos.

Con INPUT_PULLUP el botón suelto marca 1. Al presionarlo pasa a 0 porque se conecta a GND. Entonces sería FALLING si quiero que se active cuando lo tocan.

Para cuando lo sueltan = RISING.

Sin una resistencia que fije el nivel se puede quedar flotante. El pin lee cosas que nadie hizo.

GPIO 34 al 39 solamente entrada y no tienen pull-up. En el ejemplo del 35 se necesitaba una resistencia externa a 3,3V (10k).

Pines 6 al 11 no usarlos, son de la Flash del módulo. Los strapping se revisan cuando enciende el ESP32. Si se pone un nivel que no corresponde puede que no arranque.

21 para el botón, por ejemplo.

pinMode(21, INPUT_PULLUP);

attachInterrupt(digitalPinToInterrupt(21), isrBoton, FALLING);

Esto se coloca en setup una sola vez. El nombre isrBoton sin los paréntesis, porque aquí se indica qué función usar cuando llegue el aviso.

En la ISR no hacer cosas que se demoren. Ni Serial.println, ni delay. El watchdog puede reiniciar el ESP32 cuando se queda ocupado demasiado rato.

La ISR más bien deja una señal y sale. El resto se hace en loop, como prender el LED o mostrar algo en el monitor.

La Flash guarda el programa aunque apague el ESP32. SRAM para lo que se usa mientras funciona. IRAM es para instrucciones y DRAM para datos.

Caché: la CPU no tiene que ir a buscar todo a la Flash cada vez. Pero cuando el ESP32 escribe en Flash a veces se desactiva la caché un momento.

Si en ese momento ocurre una interrupción y la ISR estaba en Flash IRAM_ATTR para que la función esté en la RAM de instrucciones. Igual tiene que ser seguro todo lo que use dentro, no solamente poner IRAM_ATTR.

volatile

El compilador puede tener una copia de una variable en un registro de la CPU. Si la ISR modifica el dato, el loop podría quedarse con el anterior.

volatile bool alerta = false;

La ISR pone alerta = true

En loop reviso esa bandera, la bajo y recién hago lo demás. No imprimir directamente desde la ISR.

La bandera no sirve para contar. Si pasan tres toques mientras está ocupado, queda un solo true. Para contar eventos se habló de contador o cola.

volatile no quiere decir que todo se haga de forma atómica, por ejemplo al sumar un contador o cuando el loop y la ISR lo cambian a la vez.

Rebote del pulsador

Aunque presione una vez, el contacto puede pasar varias veces de 1 a 0 muy rápido.

Entonces la interrupción se ejecuta más de una vez.

Antirrebote. En clase se usó 50 ms.

Se compara cuánto pasó desde el toque anterior, pero hay que tener cuidado con la función de tiempo dentro de la ISR.

Botón → GPIO detecta el flanco → ISR → bandera. Ya en el loop se atiende.

Si quiero alarma apenas se presiona con pull-up: FALLING. Si no hay antirrebote, 10 pulsaciones no necesariamente producen 10 avisos.

En el repaso del timer: 1 MHz = 1 microsegundo cada tick. Con 500000 y autorrecarga da medio segundo, o sea 2 veces por segundo.